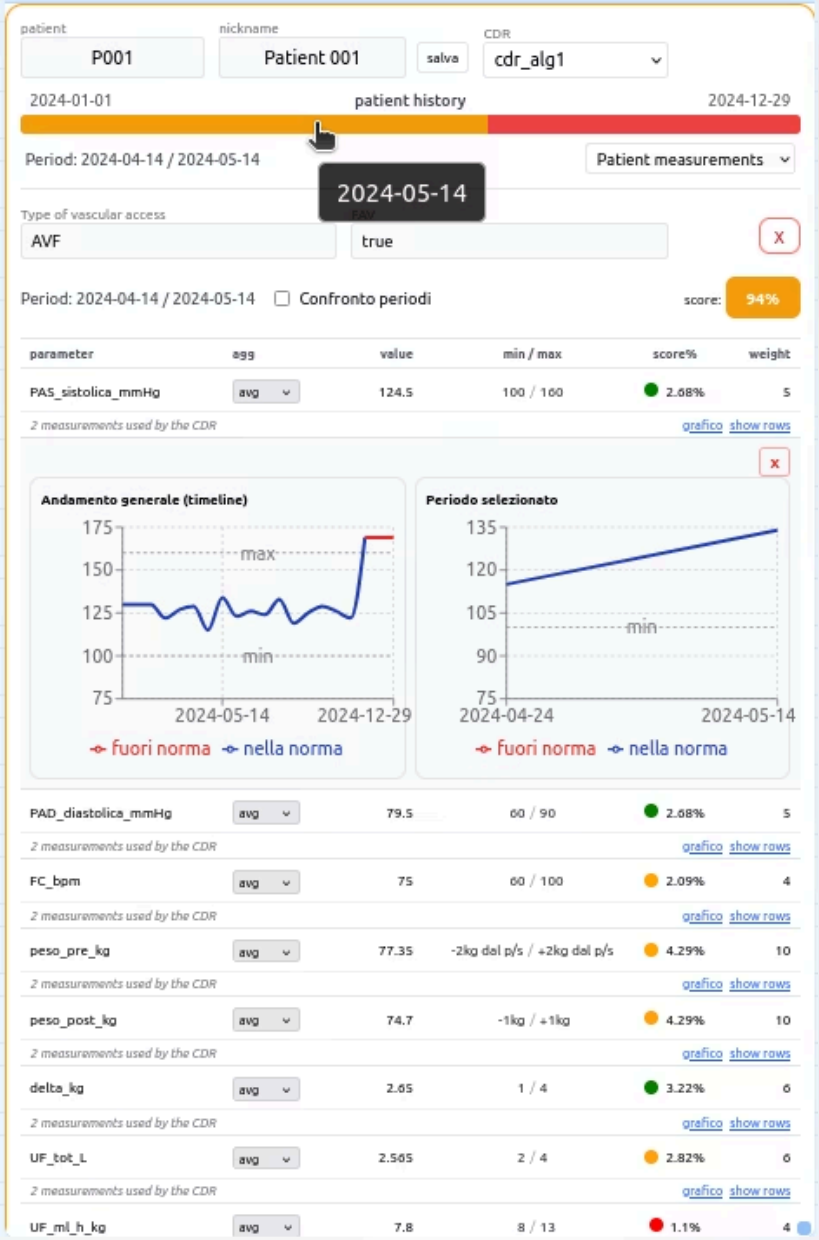
PORTFOLIO PROJECT

Clinical data is everywhere. Understanding it is the real challenge.

Ogni paziente genera nel tempo numerose misurazioni cliniche.
Il vero valore dei dati emerge solo quando possiamo **analizzarli nel contesto temporale**.

Questo progetto presenta un **widget React interattivo** progettato per analizzare parametri clinici, confrontarli con intervalli di riferimento e visualizzare l'evoluzione dello stato del paziente.



Perché Questo Progetto

I pazienti in emodialisi generano continuamente parametri clinici che devono essere letti nel loro contesto temporale per avere significato diagnostico. Un singolo valore isolato non racconta la storia completa: serve un sistema in grado di confrontare i dati con intervalli di riferimento, evidenziare anomalie e restituire una valutazione sintetica dello stato del paziente.

Questo progetto nasce esattamente da questa esigenza: trasformare dati clinici strutturati in una interfaccia leggibile e clinicamente significativa.

Contesto del Progetto: Il Sistema SIATE

Questo widget è parte di un ecosistema clinico più ampio denominato **SIATE** — un sistema modulare progettato per l'analisi dei parametri clinici di pazienti in emodialisi. Il sistema prevede più widget specializzati che operano su dati condivisi, ognuno focalizzato su un dominio clinico specifico.

🧩 SIATE System

Piattaforma clinica modulare di riferimento

👤 Patient Widget

Analisi parametri e score del paziente

📝 Notes Widget

Gestione annotazioni cliniche

⚙️ Clinical Data Services

Servizi condivisi per l'accesso ai dati

Il Mio Contributo nel Progetto

Cosa ho sviluppato

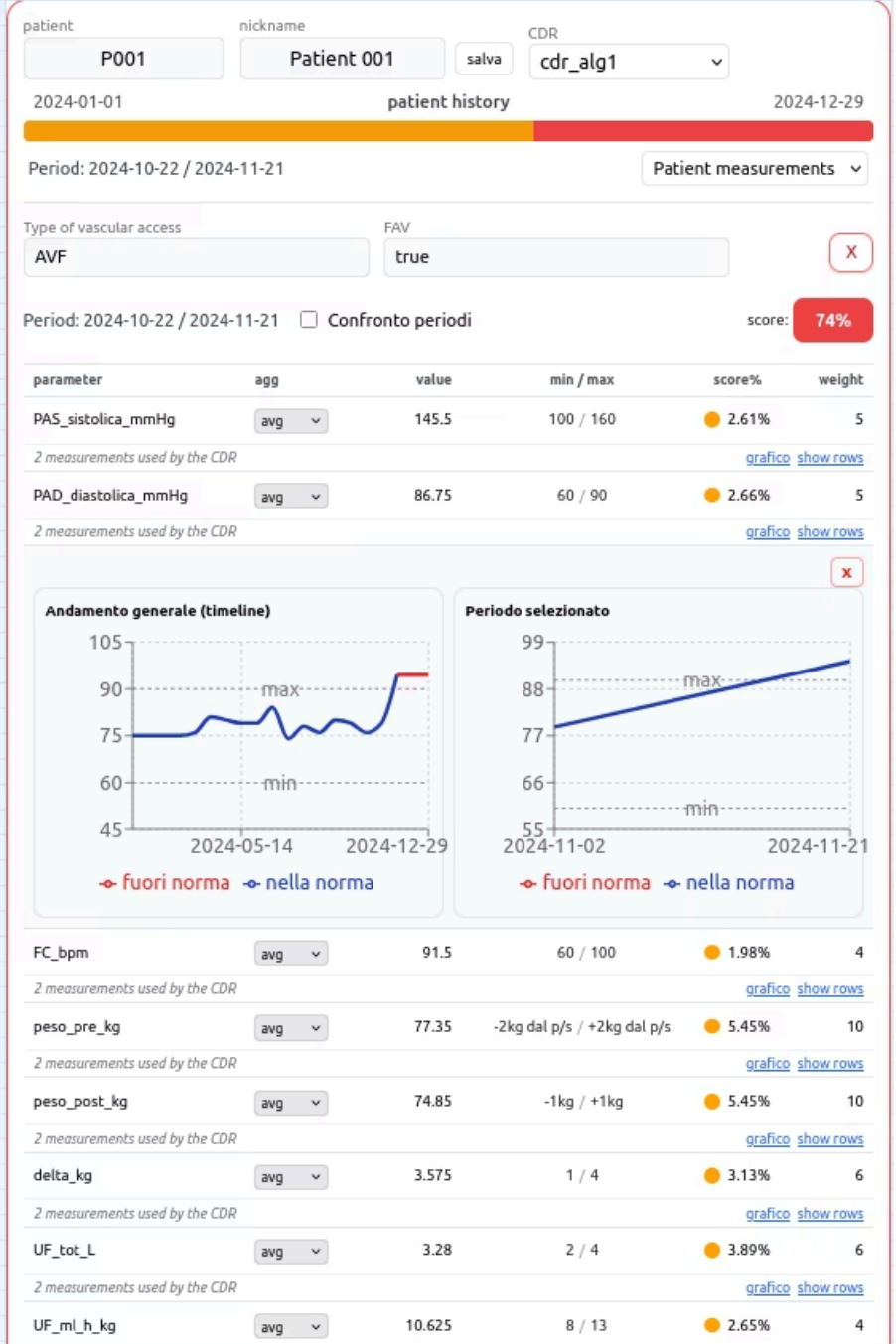
Questo repository contiene la parte del sistema interamente progettata e sviluppata da me, in modo indipendente e completo.

- Sviluppo completo del Patient Widget
- Backend Python di supporto
- Integrazione frontend–backend
- Sviluppo condiviso della canvas interattiva

Patient Widget
Interfaccia React per l'interazione paziente

Backend Support
API Flask e logica di supporto

Canvas Integration
Dashboard modulare e integrazione UI



Il Patient Widget: Il Cuore dell'Interfaccia

Il widget paziente è il componente centrale del sistema. Consente di visualizzare i parametri clinici di un paziente, confrontarli con gli intervalli di riferimento e ottenere uno **score clinico aggregato** che sintetizza la stabilità complessiva del paziente in un dato periodo.

📊 Parametri

Visualizzazione valori clinici con range min/max

🎯 Score

Percentuale aggregata di stabilità clinica

📅 Timeline

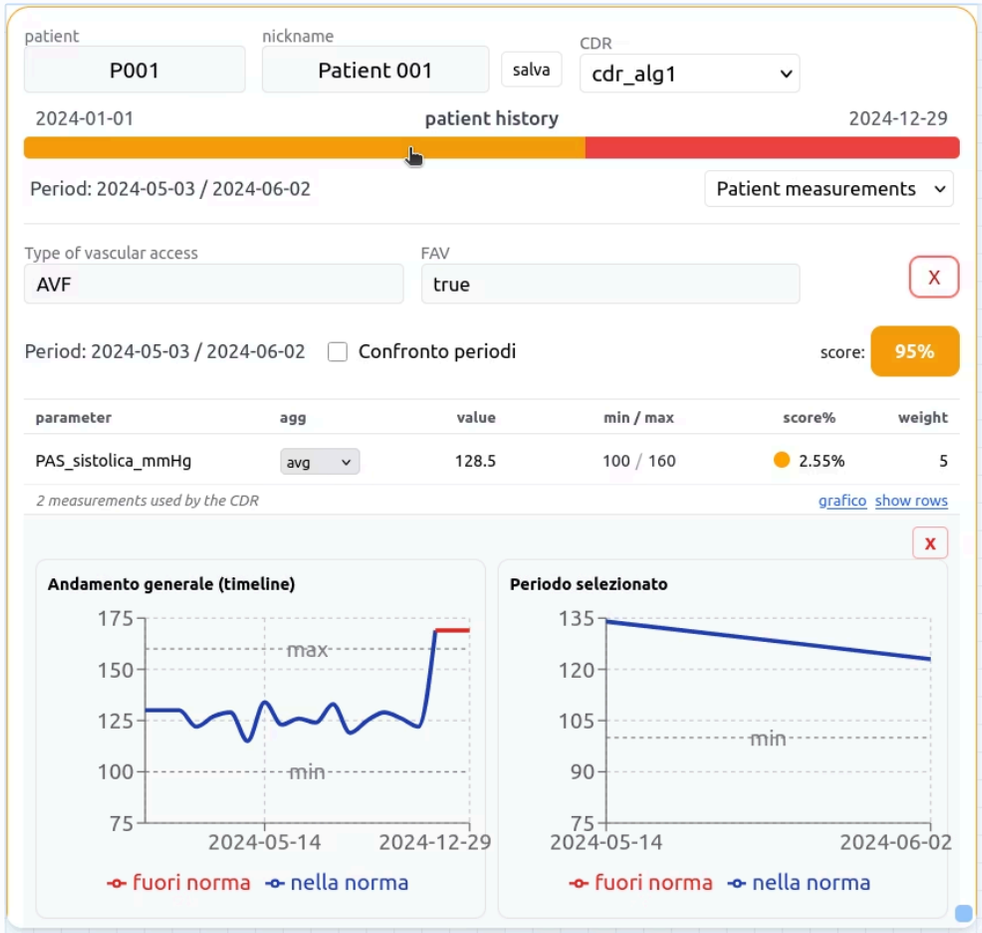
Andamento temporale dei parametri nel periodo

🔍 Confronto

Comparazione tra periodi temporali diversi

Analisi Temporale dei Parametri

I dati clinici acquisiscono significato reale solo se letti nel tempo. Il widget implementa una logica di analisi time-series che permette di visualizzare l'andamento di ogni parametro, confrontare periodi diversi e identificare immediatamente i valori fuori norma attraverso una codifica visiva intuitiva.



Andamento generale

Timeline completa dell'intero storico del paziente

Periodo selezionato

Zoom su un intervallo temporale specifico per analisi di dettaglio

Fuori norma / Nella norma

Codifica cromatica immediata per identificare anomalie cliniche

Come Viene Calcolato lo Score Clinico

Lo score clinico non è un semplice conteggio: è il risultato di una **logica di valutazione basata su regole**. Ogni parametro clinico viene verificato rispetto al proprio range di normalità e contribuisce al punteggio finale con un peso specifico, configurabile per ogni CDR (Clinical Decision Rule). Il risultato è espresso come percentuale di stabilità clinica del paziente nel periodo analizzato.

Architettura Generale del Sistema

Il sistema è strutturato in tre livelli ben separati che comunicano tramite API REST. Questa separazione garantisce manutenibilità, testabilità e la possibilità di sostituire o evolvere ciascun livello indipendentemente dagli altri.

1	2	3	4
Frontend	**API REST**	**Backend**	**Data Layer**
React — interfaccia utente, visualizzazione parametri, gestione stato	Contratto di comunicazione tra client e server	Flask (Python) — logica applicativa, calcolo score, gestione dati	Dataset JSON con misurazioni cliniche simulate

Perché Ho Creato un Backend di Supporto

Il problema

Il sistema originale SIATE non forniva un backend accessibile per lo sviluppo e il testing del widget in isolamento. Il componente richiedeva però dati dinamici, realistici e strutturati per funzionare correttamente.

La soluzione

Ho sviluppato un backend Python minimale con Flask, capace di simulare il comportamento del sistema reale e di esporre le API necessarie al widget.

Struttura del Repository

Organizzazione del codice

```
project/
├── backend/
│   ├── app.py
│   ├── routes/
│   ├── logic/
│   └── data/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── context/
│   │   └── services/
│   └── public/
└── README.md
```

Principi di progettazione

Separazione netta

Frontend e backend sono moduli indipendenti con responsabilità distinte

Struttura modulare

Ogni componente è isolato e riusabile, facilitando l'estensione futura

Scalabilità

L'architettura è pensata per aggiungere nuovi widget e servizi senza refactoring

Backend: Servizi e Logica Applicativa

Il backend è sviluppato in **Python con Flask** e costituisce il layer applicativo del sistema. Espone endpoint REST consumati dal widget e implementa la logica di valutazione dei parametri clinici, incluso il calcolo dello score aggregato.

API Endpoints

Rotte REST per fornire dati clinici paginati per paziente e periodo temporale

Gestione Dataset

Lettura e parsing di dataset JSON con misurazioni cliniche strutturate

Logica di Valutazione

Modulo dedicato al calcolo dello score: verifica range, applicazione pesi, aggregazione

Frontend: Architettura React

L'interfaccia utente è sviluppata interamente con **React**, seguendo un'architettura a componenti modulari. La gestione dello stato è centralizzata tramite Context API, mentre i servizi API sono incapsulati in moduli dedicati per garantire separazione delle responsabilità.

Components

Widget, grafici, tabelle parametri — tutti riusabili e isolati

Context API

Stato globale del paziente condiviso tra componenti

Services

Layer di comunicazione con il backend via fetch/axios

Tecnologie Utilizzate

1 React + JavaScript

- Architettura a componenti modulari
- Context API per gestione stato
- Hook personalizzati per logica riusabile

2 Python + Flask

- REST API lightweight e rapida da configurare
- Logica di business per calcolo score
- Gestione e parsing dataset JSON

3 JSON Data Layer

- Misurazioni cliniche simulate
- Struttura dati reale e documentata
- Facilmente sostituibile con DB reale

Uso dell'AI nello Sviluppo

AI come strumento professionale

L'intelligenza artificiale è stata integrata nel flusso di lavoro come **acceleratore di sviluppo**, non come sostituto del ragionamento tecnico. Ogni soluzione generata è stata compresa, valutata e adattata al contesto specifico del progetto.

→ Progettazione tecnica

Supporto nella definizione dell'architettura e delle interfacce API

→ Debugging assistito

Analisi degli errori e identificazione rapida delle cause radice

→ Code review

Revisione del codice per migliorare leggibilità e coerenza

→ Sviluppo iterativo

Cicli rapidi di prototipazione, test e raffinamento continuo

Esplora il Progetto su GitHub

Il repository è pubblico e include tutto il codice sorgente, la struttura frontend/backend e la documentazione necessaria per eseguire il widget in locale. È possibile esplorare ogni layer dell'architettura e testare il sistema con il dataset simulato incluso.

📁 Codice completo

Frontend React e backend Flask interamente disponibili

📄 Documentazione

README dettagliato con istruzioni di setup e architettura

🧪 Dataset simulato

Dati clinici di esempio per testare il widget immediatamente

Cosa Dimostra Questo Progetto

Questo progetto è la prova concreta di un insieme di competenze tecniche e progettuali applicate a un dominio reale e complesso come quello clinico. Non è un tutorial né un esercizio accademico: è un sistema funzionante, progettato con attenzione all'architettura e all'esperienza utente.

Frontend modulare

Architettura React scalabile con componenti riusabili e separazione delle responsabilità

Integrazione full-stack

Progettazione e implementazione dell'intera catena frontend → API → backend → data

Gestione dati clinici

Analisi time-series, verifica range, calcolo score aggregato su dati strutturati reali

Architettura software

Progettazione di sistemi modulari, espandibili e manutenibili nel tempo